My Templates

Bigragh

Bigraph Match

```cpp
//
// Created by 24087 on 24-11-14.
//
#include <algorithm>
#include <climits>
#include <iostream>
#include <queue>
#include <unordered_map>
#include <vector>
#include <limits>

class Dinic {
    struct Edge {
        int to;
        long long capacity, flow;
        Edge(int to, long long capacity) : to(to), capacity(capacity), flow(0) {}
    };

    int n, source, sink;
    std::vector<std::vector<int>> adj;    // 邻接表存储边的索引
    std::vector<Edge> edges;              // 存储边的信息
    std::vector<int> level;               // 分层图
    std::vector<int> ptr;                 // 当前弧优化的指针
    const long long INF = std::numeric_limits<long long>::max(); // 无限大表示

    bool bfs() {
        std::queue<int> q;
        level.assign(n + 1, -1);           // 初始化分层图，从1开始
        level[source] = 0;
        q.push(source);

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int idx : adj[u]) {
                const Edge& e = edges[idx];
                if (e.flow < e.capacity && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }

        return level[sink] != -1;          // 是否能够到达汇点
    }

    long long dfs(int u, long long pushed) {
        if (u == sink) return pushed;
```

```
        for (int& i = ptr[u]; i < adj[u].size(); ++i) {
            int idx = adj[u][i];
            Edge& e = edges[idx];
            if (level[u] + 1 != level[e.to] || e.flow == e.capacity) continue;
            long long flow = dfs(e.to, std::min(pushed, e.capacity - e.flow));
            if (flow > 0) {
                e.flow += flow;
                edges[idx ^ 1].flow -= flow;  // 更新反向边
                return flow;
            }
        }
        return 0;
    }

public:
    Dinic(int n, int source, int sink) : n(n), source(source), sink(sink) {
        adj.resize(n + 1);    // 从1开始，多分配一个位置
        level.resize(n + 1);
        ptr.resize(n + 1);
    }

    void add_edge(int u, int v, long long capacity) {
        edges.emplace_back(v, capacity);    // 正向边
        edges.emplace_back(u, 0);           // 反向边，容量为0
        adj[u].push_back(edges.size() - 2); // 正向边的索引
        adj[v].push_back(edges.size() - 1); // 反向边的索引
    }

    long long max_flow() {
        long long total_flow = 0;
        while (bfs()) {
            ptr.assign(n + 1, 0);           // 每次BFS后重置指针
            while (const long long flow = dfs(source, INF)) {
                total_flow += flow;
            }
        }
        return total_flow;
    }

    void debug_print_flow() {
        for (int i = 1; i <= n; i++) {
            for (int idx : adj[i]) {
                const Edge& e = edges[idx];
                std::cerr << i << "->" << e.to << " " << edges[idx].capacity << std::endl;
            }
        }
    }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);
```

```
    int n, m, e;
    std::cin >> n >> m >> e;

    Dinic dinic(n + m + 2, n + m + 1, n + m + 2);

    for (int i = 1; i <= n; i++) {
        dinic.add_edge(n + m + 1, i, 1);
    }
    for(int i = 1; i <= m; i++) {
        dinic.add_edge(n + i, n + m + 2, 1);
    }
    for(int i = 1, u, v; i <= e; i++) {
        std::cin >> u >> v;
        dinic.add_edge(u, n + v, 1);
    }

    std::cout << dinic.max_flow() << std::endl;

}
```

Bigragh Weight Match

```
//
// Created by 24087 on 24-11-14.
//
#include <climits>
#include <iostream>
#include <map>
#include <queue>
#include <vector>

#define int long long

class MinCostMaxFlow {
    const int INF = LLONG_MAX;

   public:
    explicit MinCostMaxFlow(int n)
        : n(n), adj(n + 1), dis(n + 1), vis(n + 1), cur(n + 1), ret(0) {}

    void add_edge(int u, int v, int w, int c) {
        adj[u].emplace_back(Edge{v, w, c, static_cast<int>(adj[v].size()), true});
        adj[v].emplace_back(
            Edge{u, 0, -c, static_cast<int>(adj[u].size()) - 1, false});
    }

    int min_cost_max_flow(int s, int t) {
        int max_flow = 0;
        while (spfa(s, t)) {
            std::fill(vis.begin(), vis.end(), false);
            int flow;
```

```
        while ((flow = dfs(s, t, INF))) {
            max_flow += flow;
        }
    }
    return max_flow;
}

int get_cost() const { return ret; }

void debug_print_flows() {
    for (int i = 1; i <= n; i++) {
        for (auto &edge : adj[i]) {
            std::cerr << i << "->" << edge.to << ": "
                      << adj[edge.to][edge.rev].cap << '\n';
        }
    }
}

void print_ans() {
    std::map<int, int> mp;
    int nn = (n - 2) / 2;
    // std::cerr << nn << std::endl;
    for (int i = 1; i <= nn; i++) {
        for (auto &edge : adj[i]) {
            if (edge.is_positive && edge.cap == 0) {
                mp[edge.to - nn] = i;
            }
        }
    }
    for (int i = 1; i <= nn; i++) {
        // if (mp[i] == 0) {
        //     continue;
        // }
        std::cout << mp[i] << ' ';
    }
}

private:
 struct Edge {
    int to, cap, cost, rev;
    bool is_positive;
 };

 int n, ret;
 std::vector<std::vector<Edge>> adj;
 std::vector<int> dis, cur;
 std::vector<bool> vis;

 bool spfa(int s, int t) {
    std::fill(dis.begin(), dis.end(), INF);
    dis[s] = 0;
    std::queue<int> q;
    q.push(s);
```

```
        vis[s] = true;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            vis[u] = false;

            for (const auto &e : adj[u]) {
                if (e.cap > 0 && dis[e.to] > dis[u] + e.cost) {
                    dis[e.to] = dis[u] + e.cost;
                    if (!vis[e.to]) {
                        q.push(e.to);
                        vis[e.to] = true;
                    }
                }
            }
        }
        return dis[t] != INF;
    }

    int dfs(int u, int t, int flow) {
        if (u == t) return flow;
        vis[u] = true;
        int total_flow = 0;

        for (auto &e : adj[u]) {
            if (!vis[e.to] && e.cap > 0 && dis[e.to] == dis[u] + e.cost) {
                int pushed = dfs(e.to, t, std::min(flow - total_flow, e.cap));
                if (pushed > 0) {
                    e.cap -= pushed;
                    adj[e.to][e.rev].cap += pushed;
                    ret += pushed * e.cost;
                    total_flow += pushed;
                    if (total_flow == flow) break;
                }
            }
        }
        vis[u] = false;
        return total_flow;
    }
};

signed main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m;
    std::cin >> n >> m;

    MinCostMaxFlow mcmf(n * 2 + 2);

    for (int i = 1; i <= n; i++) {
```

```
        mcmf.add_edge(2 * n + 1, i, 1, 0);
        //mcmf.add_edge(i, 2 * n + 2, 1, 0);
        mcmf.add_edge(i + n, 2 * n + 2, 1, 0);
    }

    for (int i = 1, u, v, w; i <= m; i++) {
        std::cin >> u >> v >> w;
        mcmf.add_edge(u, v + n, 1, -w);
    }

    auto flow = mcmf.min_cost_max_flow(2 * n + 1, 2 * n + 2);
    auto cost = -mcmf.get_cost();

    //std::cerr << flow << '\n';
    std::cout << cost << '\n';

    mcmf.print_ans();
}
```

Binary Index Tree

```
class BIT {
    using ll = long long;
    int n;
    std::vector<ll> tr;
    constexpr static int low_bit(const int x) {
        return x & -x;
    }

public:
    BIT(int n): n(n), tr(n + 1) {}
    void add(int idx, ll val) {
        for(int i = idx; i <= n; i += low_bit(i)) {
            tr[i] += val;
        }
    }
    ll query(int idx) {
        ll res = 0;
        for(int i = idx; i; i -= low_bit(i)) {
            res += tr[i];
        }
        return res;
    }
    ll query(int l, int r) {
        return query(r) - query(l - 1);
    }
};

class EBIT {
    class BIT {
        using ll = long long;
        int n;
```

```
        std::vector<ll> tr;
        constexpr static int low_bit(const int x) {
            return x & -x;
        }

    public:
        explicit BIT(int n): n(n), tr(n + 1) {}
        void add(int idx, ll val) {
            for(int i = idx; i <= n; i += low_bit(i)) {
                tr[i] += val;
            }
        }
        ll query(int idx) {
            ll res = 0;
            for(int i = idx; i; i -= low_bit(i)) {
                res += tr[i];
            }
            return res;
        }
        ll query(int l, int r) {
            return query(r) - query(l - 1);
        }
    };

    int n;
    BIT d, di;
public:
    explicit EBIT(int n): n(n), d(n), di(n) {}
    //sum(r) = d_sum * (r + 1) - di_sum
    using ll = long long;

    ll query(int idx) {
        return d.query(idx) * (idx + 1) - di.query(idx);
    }
    ll query(int l, int r) {
        return query(r) - query(l - 1);
    }

    void add(int l, int r, int val) {
        d.add(l, val);
        d.add(r + 1, -val);

        di.add(l, val * l);
        di.add(r + 1, -val * (r + 1));
    }
    void add(int idx, int val) {
        add(idx, idx, val);
    }
};
```

Binary Search

```cpp
int binary_search(int destination, const std::vector<int> &vec) {
    int l = 0, r = static_cast<int>(vec.size()) - 1;
    // 全都不满足条件，归0，区间外
    //全都满足就是最后一个，区间内
    while(l < r) {
        int mid = (l + r + 1) / 2;//attention
        if(vec[mid] < destination) {
            l = mid;
        } else {
            r = mid - 1;
        }
    }
    return r;
}
//此方法是在讲，期望l，r都指向满足条件的最后一个

int binary_search(int destination, const std::vector<int> &vec) {
    int l = 1, r = static_cast<int>(vec.size());
    // 全都不满足条件，归1，区间内
    //全都满足就是最后一个 + 1，区间外
    while(l < r) {
        int mid = (l + r) / 2;//attention
        if(vec[mid] < destination) {
            l = mid + 1;
        } else {
            r = mid;
        }
    }
    return r;
}
//此方法是在讲，期望l，r都指向不满足条件的第一个

int binary_search(int destination, const std::vector<int> &vec) {
    int l = 0, r = static_cast<int>(vec.size());//n + 1
    while(l + 1 < r) {
        int mid = (l + r) / 2;
        if(vec[mid] < destination) {
            l = mid;
        } else {
            r = mid;
        }
    }
    return r;
}
//此方法是在讲期望l指向满足条件最后一个，r期望指向不满足条件第一个
```

Chtholly Tree

```cpp
long long pow(long long a, long long b, int p) {
    a %= p;//!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
    long long res = 1;
    while(b) {
```

```
        if(b & 1) {
            res = res * a % p;
        }
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

class ChthollyTree {
    struct Node {
        int l, r;
        mutable long long val;
        Node(int l, int r, long long val) : l(l), r(r), val(val) {}
        bool operator<(const Node &x) const { return l < x.l; }
    };
    int n;
    std::set<Node> s;
    auto split(int x) {
        auto it = s.lower_bound(Node(x, 0, 0));
        if (it != s.end() && it->l == x) {
            return it;
        }
        --it;
        int l = it->l, r = it->r;
        long long val = it->val;
        s.erase(it);
        s.emplace(l, x - 1, val);
        return s.emplace(x, r, val).first;
    }

   public:
    explicit ChthollyTree(const int n) : n(n) {
        s.emplace(1, n, 0);  // 不用多插入一位
    }

    void assign(int l, int r, long long v) {
        const auto itr = split(r + 1);
        const auto itl = split(l);  // 先右后左，防止左边界迭代器失效
        s.erase(itl, itr);
        s.emplace(l, r, v);
    }

    void add(int l, int r, long long v) {
        for (auto itr = split(r + 1), itl = split(l); itl != itr; ++itl) {
            itl -> val += v;
        }
    }
    long long query_power(int l, int r,int x, int p) {
        long long res = 0;
        for (auto itr = split(r + 1), itl = split(l); itl != itr; ++itl) {
            (res += pow(itl->val, x, p) * (itl->r - itl->l + 1)) %= p;
        }
```

```
        return res;
    }
    long long get_x_th(int l, int r, int x) {
        std::map<long long, int> mp;
        for (auto itr = split(r + 1), itl = split(l); itl != itr; ++itl) {
            mp[itl->val] += itl->r - itl->l + 1;
        }
        for (const auto & [fst, snd] : mp) {
            if (x <= snd) {
                return fst;
            }
            x -= snd;
        }
        return -1;
    }
};
```

Disjoint Set Union

```
class DisjointSetUnion {
    int n;
    std::vector<int> fa, size;

public:
    explicit DisjointSetUnion(const int _n): n(_n), fa(_n + 1), size(_n + 1, 1) {
        std::iota(fa.begin(), fa.end(), 0);
    }
    int find(const int x) {
        if(fa[x] == x)
            return x;
        return fa[x] = find(fa[x]);
    }
    void unite(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y)
            return;
        if(size[x] < size[y])
            std::swap(x, y);
        fa[y] = x;
        size[x] += size[y];
    }
};
```

Tree Decomposition

```
class TreeDecomposition {
    struct Node {
        int fa = 0, siz = 1, dep = 0, son = 0, top = 0, dfn = 0;
    };
```

```
    int n, s, mod;
    std::vector<std::vector<int>> tr;
    std::vector<Node> nodes;
    Segment_tree<ll> bit;

    void dfs1(int now, int father, int depth) {
        nodes[now].fa = father;
        nodes[now].dep = depth;
        for (int nxt : tr[now]) {
            if (nxt == father) {
                continue;
            }
            dfs1(nxt, now, depth + 1);
            nodes[now].siz += nodes[nxt].siz;
            if (nodes[nxt].siz >
                nodes[nodes[now].son].siz) {  // 要保证nodes[0].siz = 0
                nodes[now].son = nxt;
            }
        }
    }

    void dfs2(int now, int tp) {
        static int tot = 0;
        nodes[now].top = tp;
        nodes[now].dfn = ++tot;
        if (nodes[now].son == 0) return;
        dfs2(nodes[now].son, tp);
        for (int nxt : tr[now]) {
            if (nxt == nodes[now].son || nxt == nodes[now].fa) continue;
            dfs2(nxt, nxt);
        }
    }

   public:
    TreeDecomposition(int n, int s, int mod, std::vector<std::vector<int>> &&tr,
                      std::vector<int> &&val)
        : n(n), s(s), mod(mod), tr(std::move(tr)), nodes(n + 1), bit(n, mod) {
        nodes[0].siz = 0;  //!!!!!!!
        dfs1(s, 0, 1);
        dfs2(s, s);
        for (int i = 1; i <= n; i++) {
            bit.add(nodes[i].dfn, nodes[i].dfn, val[i]);
        }
    }

    void modify_lca(int u, int v, int val) {
        while (nodes[u].top != nodes[v].top) {
            if (nodes[nodes[u].top].dep > nodes[nodes[v].top].dep) {
                bit.add(nodes[nodes[u].top].dfn, nodes[u].dfn, val);
                u = nodes[nodes[u].top].fa;
            } else {
                bit.add(nodes[nodes[v].top].dfn, nodes[v].dfn, val);
                v = nodes[nodes[v].top].fa;
```

```
            }
        }
        bit.add(nodes[u].dfn, nodes[v].dfn, val);
    }

    void modify_subtree(int u, int val) {
        bit.add(nodes[u].dfn, nodes[u].dfn + nodes[u].siz - 1, val);
    }

    ll query_lca(int u, int v) {
        ll res = 0;
        while (nodes[u].top != nodes[v].top) {
            if (nodes[nodes[u].top].dep > nodes[nodes[v].top].dep) {
                res += bit.ask(nodes[nodes[u].top].dfn, nodes[u].dfn);
                res %= mod;
                u = nodes[nodes[u].top].fa;
            } else {
                res += bit.ask(nodes[nodes[v].top].dfn, nodes[v].dfn);
                res %= mod;
                v = nodes[nodes[v].top].fa;
            }
        }
        res += bit.ask(nodes[u].dfn, nodes[v].dfn);
        res %= mod;
        return res;
    }
    ll query_subtree(int u) {
        return bit.ask(nodes[u].dfn, nodes[u].dfn + nodes[u].siz - 1);
    }
};
```

Monotone Queue

```
#include <iostream>
#include <deque>
#include <vector>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, k;
    std::cin >> n >> k;
    std::vector<int> vec(n + 1);
    std::deque<int> dq_max, dq_min;

    for(int i = 1; i <= n; i++) {
        std::cin >> vec[i];
    }
    for(int i = 1; i <= n; i++) {
        if(!dq_min.empty() && dq_min.front() + k <= i) {
```

```
            dq_min.pop_front();
        }
        while(!dq_min.empty() && vec[dq_min.back()] >= vec[i]) {
            dq_min.pop_back();
        }
        dq_min.push_back(i);
        if(i >= k) {
            std::cout << vec[dq_min.front()] << ' ';
        }
    }
    std::cout << '\n';
    for(int i = 1; i <= n; i++) {
        if(!dq_max.empty() && dq_max.front() + k <= i) {
            dq_max.pop_front();
        }
        while(!dq_max.empty() && vec[dq_max.back()] <= vec[i]) {
            dq_max.pop_back();
        }
        dq_max.push_back(i);
        if(i >= k) {
            std::cout << vec[dq_max.front()] << ' ';
        }
    }
}
```

Monotone Stack

```
//
// Created by 24087 on 24-10-21.
//
#include <iostream>
#include <vector>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n;
    std::cin >> n;
    std::vector<int> stk, vec(n + 1);
    for(int i = 1; i <= n; ++i) {
        std::cin >> vec[i];
    }

    long long ans = 0;

    for(int i = n; i > 0; i--) {
        while(!stk.empty() && vec[stk.back()] < vec[i]) {//牛只能看到比自己矮的，所以要找第一个>=自己的
            stk.pop_back();
        }
        ans += stk.empty() ? n - i : stk.back() - i - 1;//居然看不到最高的那个牛
```

```
            stk.push_back(i);
        }
        std::cout << ans;
    }
```

Max Flow

Dinic

```
class Dinic {
    struct Edge {
        int to;
        long long capacity, flow;
        Edge(int to, long long capacity): to(to), capacity(capacity), flow(0) {}
    };

    int n, source, sink;
    std::vector<std::vector<int>> adj;    // 邻接表存储边的索引
    std::vector<Edge> edges;              // 存储边的信息
    std::vector<int> level;               // 分层图
    std::vector<int> ptr;                 // 当前弧优化的指针
    const long long INF = std::numeric_limits<long long>::max(); // 无限大表示

    bool bfs() {
        std::queue<int> q;
        level.assign(n + 1, -1);           // 初始化分层图，从1开始
        level[source] = 0;
        q.push(source);

        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(int idx : adj[u]) {
                const Edge &e = edges[idx];
                if(e.flow < e.capacity && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }

        return level[sink] != -1;          // 是否能够到达汇点
    }

    long long dfs(int u, long long pushed) {
        if(u == sink) return pushed;
        for(int &i = ptr[u]; i < adj[u].size(); ++i) {
            int idx = adj[u][i];
            Edge &e = edges[idx];
            if(level[u] + 1 != level[e.to] || e.flow == e.capacity) continue;
            long long flow = dfs(e.to, std::min(pushed, e.capacity - e.flow));
            if(flow > 0) {
```

```
                e.flow += flow;
                edges[idx ^ 1].flow -= flow;  // 更新反向边
                return flow;
            }
        }
        return 0;
    }

public:
    Dinic(int n, int source, int sink): n(n), source(source), sink(sink) {
        adj.resize(n + 1);    // 从1开始，多分配一个位置
        level.resize(n + 1);
        ptr.resize(n + 1);
    }

    void add_edge(int u, int v, long long capacity) {
        edges.emplace_back(v, capacity);    // 正向边
        edges.emplace_back(u, 0);           // 反向边，容量为0
        adj[u].push_back(edges.size() - 2); // 正向边的索引
        adj[v].push_back(edges.size() - 1); // 反向边的索引
    }

    long long max_flow() {
        long long total_flow = 0;
        while(bfs()) {
            ptr.assign(n + 1, 0);           // 每次BFS后重置指针
            while(long long flow = dfs(source, INF)) {
                total_flow += flow;
            }
        }
        return total_flow;
    }

    void debug_print_flow() {
        for(int i = 1; i <= n; i++) {
            for(int idx : adj[i]) {
                if((idx & 1) == 1) {
                    continue;
                }
                const Edge &e = edges[idx];
                std::cerr << i << "->" << e.to << " " << e.flow << std::endl;
            }
        }
    }
};


int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m, s, t;
```

```
    std::cin >> n >> m >> s >> t;

    Dinic dinic(n, s, t);

    for(int i = 0; i < m; ++i) {
        int u, v;
        long long capacity;
        std::cin >> u >> v >> capacity;
        dinic.add_edge(u, v, capacity);
    }
    std::cout << dinic.max_flow() << std::endl;

    dinic.debug_print_flow();
    return 0;
}
```

EdmondKarp

```
class EdmondsKarp {
    int n, s, t;
    std::vector<std::unordered_map<int, long long>> graph;  // residual graph
    std::vector<int> fa;

    long long bfs() {
        std::fill(fa.begin(), fa.end(), -1);
        std::queue<std::pair<int, long long>> q;
        q.emplace(s, LLONG_MAX);
        fa[s] = s;

        while (!q.empty()) {
            auto [now, flow] = q.front();
            q.pop();

            for (auto &[to, cap] : graph[now]) {
                if (fa[to] == -1 && cap > 0) {  // 没访问过，且容量大于 0
                    fa[to] = now;
                    auto new_flow = std::min(flow, cap);
                    if (to == t) {
                        int cur = t;
                        while (cur != s) {
                            int prev = fa[cur];
                            graph[prev][cur] -= new_flow;
                            graph[cur][prev] += new_flow;
                            cur = prev;
                        }
                        return new_flow;
                    }
                    q.emplace(to, new_flow);
                }
            }
        }
        return 0;
```

```
    }

public:
    EdmondsKarp(int n, int s, int t)
        : n(n), s(s), t(t), fa(n + 1), graph(n + 1) {}

    void add_edge(int u, int v, int w) {
        graph[u][v] += w;
    }

    long long max_flow() {
        long long total_flow = 0;

        long long new_flow;
        while ((new_flow = bfs())) {
            total_flow += new_flow;
        }
        return total_flow;
    }
};




                    w += flow;
            }
        }
        return total_flow;
    }
};
```

FordFulkerson

```
class EdmondsKarp {
    int n, s, t;
    std::vector<std::unordered_map<int, int>> graph;  // residual graph
    std::vector<bool> vis;

    int dfs(int now, int flow) {
        if (now == t) {
            return flow;
        }
        vis[now] = true;
        for (auto [to, val] : graph[now]) {  // 找到一条路即可
            if (vis[to] || val == 0) continue;//一定要判断无效边啊！！vis等着你
            int new_flow = dfs(to, std::min(flow, val));
            if (new_flow > 0) {
                graph[now][to] -= new_flow;
                graph[to][now] += new_flow;
                return new_flow;
            }
        }
```

```
        return 0;
    }

   public:
    EdmondsKarp(int n, int s, int t,
                std::vector<std::unordered_map<int, int>> &&graph)
        : n(n), s(s), t(t), vis(n + 1), graph(std::move(graph)) {}

    long long max_flow() {
        long long ret = 0;
        int tmp;
        do {
            std::fill(vis.begin(), vis.end(), false);  // 每次都需要重新标记访问
            tmp = dfs(s, INT_MAX);
            ret += tmp;
        } while (tmp != 0);
        return ret;
    }
};
```

Min Cost Max Flow

DinicSSP

```
class MinCostMaxFlow {
    const int INF = INT_MAX;
public:
    explicit MinCostMaxFlow(int n) : n(n), adj(n + 1), dis(n + 1), vis(n + 1), cur(n + 1), ret(0) {}

    void add_edge(int u, int v, int w, int c) {
        adj[u].emplace_back(Edge{v, w, c, static_cast<int>(adj[v].size()), true});
        adj[v].emplace_back(Edge{u, 0, -c, static_cast<int>(adj[u].size()) - 1, false});
    }

    int min_cost_max_flow(int s, int t) {
        int max_flow = 0;
        while (spfa(s, t)) {
            std::fill(vis.begin(), vis.end(), false);
            int flow;
            while ((flow = dfs(s, t, INF))) {
                max_flow += flow;
            }
        }
        return max_flow;
    }

    int get_cost() const { return ret; }

    void debug_print_flows() {
        for (int i = 1; i <= n; i++) {
            for(auto &edge : adj[i]) {
                if(edge.is_positive)
```

```
                    std::cerr << i << "->" << edge.to << ": " << adj[edge.to][edge.rev].cap << '\n';
            }
        }
    }

private:
    struct Edge {
        int to, cap, cost, rev;
        bool is_positive;
    };

    int n, ret;
    std::vector<std::vector<Edge>> adj;
    std::vector<int> dis, cur;
    std::vector<bool> vis;

    bool spfa(int s, int t) {
        std::fill(dis.begin(), dis.end(), INF);
        dis[s] = 0;
        std::queue<int> q;
        q.push(s);
        vis[s] = true;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            vis[u] = false;

            for (const auto &e : adj[u]) {
                if (e.cap > 0 && dis[e.to] > dis[u] + e.cost) {
                    dis[e.to] = dis[u] + e.cost;
                    if (!vis[e.to]) {
                        q.push(e.to);
                        vis[e.to] = true;
                    }
                }
            }
        }
        return dis[t] != INF;
    }

    int dfs(int u, int t, int flow) {
        if (u == t) return flow;
        vis[u] = true;
        int total_flow = 0;

        for (auto &e : adj[u]) {
            if (!vis[e.to] && e.cap > 0 && dis[e.to] == dis[u] + e.cost) {
                int pushed = dfs(e.to, t, std::min(flow - total_flow, e.cap));
                if (pushed > 0) {
                    e.cap -= pushed;
                    adj[e.to][e.rev].cap += pushed;
                    ret += pushed * e.cost;
```

```
                    total_flow += pushed;
                    if (total_flow == flow) break;
                }
            }
        }
        vis[u] = false;
        return total_flow;
    }
};


int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m, s, t;
    std::cin >> n >> m >> s >> t;
    MinCostMaxFlow mcmf(n);
    for (int i = 1, u, v, w, c; i <= m; i++) {
        std::cin >> u >> v >> w >> c;
        mcmf.add_edge(u, v, w, c);
    }
    auto max_flow = mcmf.min_cost_max_flow(s, t);
    auto min_cost = mcmf.get_cost();
    mcmf.debug_print_flows();
    std::cout << max_flow << " " << min_cost << '\n';
}
```

Segment Tree

```
using ll = long long;
template <class T>
class Segment_tree {
#define out_of_range() (r < tar_l || l > tar_r)
#define in_range() (tar_l <= l && r <= tar_r)
    struct node {
        T sum = 0;
        T add_lzy = 0;
        T mul_lzy = 1;  // 初始值！！！
    };
    // node_val == origin_val * mul_lzy + add_lzy
    int n, mod;
    std::vector<node> data;

    void pull_up(unsigned idx) {
        data[idx].sum = (data[idx << 1].sum + data[idx << 1 | 1].sum) % mod;
    }

    void make_add(unsigned idx, unsigned l, unsigned r, T val) {
        (data[idx].sum += val * (r - l + 1)) %= mod;
        (data[idx].add_lzy += val) %= mod;
```

```
    }

    void make_mul(unsigned idx, T val) {
        (data[idx].sum *= val) %= mod;
        (data[idx].add_lzy *= val) %= mod;
        (data[idx].mul_lzy *= val) %= mod;
    }

    void push_down(unsigned idx, unsigned l, unsigned r) {
        auto mid = l + (r - l) / 2;
        // mul firstly and add secondly
        // new_node_val == origin_val * mul_lzy_1 * mul_lzy_2 + add_lzy *
        // mul_lzy_2
        // + add_lzy_2
        make_mul(idx << 1, data[idx].mul_lzy);
        make_add(idx << 1, l, mid, data[idx].add_lzy);

        make_mul(idx << 1 | 1, data[idx].mul_lzy);
        make_add(idx << 1 | 1, mid + 1, r, data[idx].add_lzy);

        data[idx].add_lzy = 0;
        data[idx].mul_lzy = 1;
    }

    T ask(const unsigned idx, const unsigned l, const unsigned r,
          const unsigned tar_l, const unsigned tar_r) {
        if (out_of_range()) {
            return 0;
        }
        if (in_range()) {
            return data[idx].sum;
        }
        push_down(idx, l, r);
        auto mid = l + (r - l) / 2;
        return (ask(idx << 1, l, mid, tar_l, tar_r) +
                ask(idx << 1 | 1, mid + 1, r, tar_l, tar_r)) %
               mod;
    }

    void add(const unsigned idx, const unsigned l, const unsigned r,
             const unsigned tar_l, const unsigned tar_r, const T val) {
        if (out_of_range()) {
            return;
        }
        if (in_range()) {
            make_add(idx, l, r, val);
            return;
        }
        auto mid = l + (r - l) / 2;
        push_down(idx, l, r);
        add(idx << 1, l, mid, tar_l, tar_r, val);
        add(idx << 1 | 1, mid + 1, r, tar_l, tar_r, val);
        pull_up(idx);
```

```
    }

    void mul(const unsigned idx, const unsigned l, const unsigned r,
             const unsigned tar_l, const unsigned tar_r, const T val) {
        if (out_of_range()) {
            return;
        }
        if (in_range()) {
            make_mul(idx, val);
            return;
        }
        const auto mid = l + (r - l) / 2;
        push_down(idx, l, r);
        mul(idx << 1, l, mid, tar_l, tar_r, val);
        mul(idx << 1 | 1, mid + 1, r, tar_l, tar_r, val);
        pull_up(idx);
    }

   public:
    explicit Segment_tree(const std::vector<T> &nums,
                          const int &p = 571373) noexcept
        : n(nums.size()), mod(p), data(nums.size() * 4 + 5) {
        std::function<void(unsigned, unsigned, unsigned)> build_helper =
            [&](const unsigned idx, const unsigned l, const unsigned r) {
                if (l == r) {
                    data[idx].sum = nums[l - 1] % mod;  // 减一减一啊啊啊啊
                    return;
                }
                const auto mid = l + (r - l) / 2;
                build_helper(idx << 1, l, mid);
                build_helper(idx << 1 | 1, mid + 1, r);
                pull_up(idx);
            };
        build_helper(1, 1, n);
    }

    T ask(const unsigned l, const unsigned r) { return ask(1, 1, n, l, r); }

    void add(const unsigned l, const unsigned r, T val) {
        add(1, 1, n, l, r, val);
    }

    void mul(const unsigned l, const unsigned r, T val) {
        mul(1, 1, n, l, r, val);
    }

#undef out_of_range
#undef in_range
};
```

Sparse Table

```cpp
class SparseTable {
    int n, log_n;
    std::vector<std::vector<int>> vec;

   public:
    SparseTable(int n, const std::vector<int> &arr)
        : n(n),
          log_n(std::floor(std::log2(n))),
          vec(n + 1, std::vector<int>(log_n + 1)) {
        for(int i = 1; i <= n; i++) {
            vec[i][0] = arr[i];
        }
        for(int j = 1; j <= log_n; j++) {
            for(int i = 1; i + (1 << j) - 1 <= n; i++) {
                vec[i][j] = std::max(vec[i][j - 1], vec[i + (1 << (j - 1))][j - 1]);
            }
        }
    }
    int query(int l, int r) {
        int k = std::floor(std::log2(r - l + 1));
        return std::max(vec[l][k], vec[r - (1 << k) + 1][k]);
    }
};
```

Trie

```cpp
class ZO_Trie {
    static bool get_bit(unsigned x, int i) { return (x >> i) & 1; }

    struct Node {
        std::array<size_t, 2> nxt = {0, 0};
        int is_end = 0;
        int is_suffix = 0;  // include end
    };
    static constexpr int root = 0;
    std::vector<Node> nodes;

   public:
    explicit ZO_Trie() : nodes(1) {}
    void insert(unsigned s) {
        size_t now = root;
        for (int i = 31; i >= 0; --i) {
            if (nodes[now].nxt[get_bit(s, i)] == 0) {
                nodes[now].nxt[get_bit(s, i)] = nodes.size();
                nodes.emplace_back();
            }
            now = nodes[now].nxt[get_bit(s, i)];
            nodes[now].is_suffix++;
        }
        nodes[now].is_end++;
    }
    [[nodiscard]] int count(const unsigned s) const {
```

```
        size_t now = root;
        for (int i = 31; i >= 0; --i) {
            if (nodes[now].nxt[get_bit(s, i)] == 0) {
                return 0;
            }
            now = nodes[now].nxt[get_bit(s, i)];
        }
        return nodes[now].is_suffix;
    }
    [[nodiscard]] unsigned find_max_xor(const unsigned s) const {
        size_t now = root;
        unsigned res = 0;
        for (int i = 31; i >= 0; --i) {
            if (nodes[now].nxt[!get_bit(s, i)] != 0) {
                res |= (1 << i);
                now = nodes[now].nxt[!get_bit(s, i)];
            } else {
                now = nodes[now].nxt[get_bit(s, i)];
            }
        }
        return res;
    }
};
```

```
class Trie {
    struct Node {
        std::unordered_map<char, size_t> nxt;
        int is_end = 0;
        int is_suffix = 0;//include end
    };
    static constexpr int root = 0;
    std::vector<Node> nodes;
public:
    explicit Trie() : nodes(1) {}
    void insert(const std::string &s) {
        size_t now = root;
        for (const char c : s) {
            if (!nodes[now].nxt.count(c)) {
                nodes[now].nxt[c] = nodes.size();
                nodes.emplace_back();
            }
            now = nodes[now].nxt[c];
            nodes[now].is_suffix++;
        }
        nodes[now].is_end++;
    }
    int count(const std::string &s) {
        size_t now = root;
        for (const char c : s) {
            if (!nodes[now].nxt.count(c)) {
                return 0;
            }
```

```
            now = nodes[now].nxt[c];
        }
        return nodes[now].is_suffix;
    }
};
```

LCA

Tree Decomposition

```
class LCA {
    int n, s;
    std::vector<std::vector<int>> tr;
    std::vector<int> fa, siz, dep, son, top, dfn;

    void dfs1(int now, int father, int depth) {
        fa[now] = father;
        dep[now] = depth;
        siz[now] = 1;
        son[now] = 0;
        for(int nxt : tr[now]) {
            if(nxt == father) {
                continue;
            }
            dfs1(nxt, now, depth + 1);
            siz[now] += siz[nxt];
            if(siz[nxt] > siz[son[now]]) {
                son[now] = nxt;
            }
        }
    }
    void dfs2(int now, int tp) {
        static int  tot = 0;
        top[now] = tp;
        dfn[now] = ++tot;
        if(son[now] == 0)
            return;
        dfs2(son[now], tp);
        for(int nxt : tr[now]) {
            if(nxt == son[now] || nxt == fa[now])
                continue;
            dfs2(nxt, nxt);
        }
    }

   public:
    LCA(int n, int s, std::vector<std::vector<int>> &&tr)
        : n(n),
          s(s),
          tr(std::move(tr)),
          fa(n + 1),
          siz(n + 1),
```

```
          dep(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1) {

        dfs1(s, 0, 1);
        dfs2(s, s);
    }

    int operator()(int u, int v) {
        while(top[u] != top[v]) {
            if(dep[top[u]] > dep[top[v]]) {
                u = fa[top[u]];
            } else {
                v = fa[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }

};
```

Tarjan

```
class DisjointSetUnion {
    std::vector<int> fa, siz;

   public:
    explicit DisjointSetUnion(const int n) : fa(n + 1), siz(n + 1, 1) {
        std::iota(fa.begin(), fa.end(), 0);
    }

    int find(const int x) {
        if (fa[x] == x) return x;
        return fa[x] = find(fa[x]);
    }

    void merge(int x, int y) {//不能优化
        x = find(x);
        y = find(y);
        if (x == y) return;
        fa[y] = x;
    }
};

class LCA {
    int n, m, s;
    std::vector<std::vector<int>> tr;
    std::vector<std::vector<std::pair<int, int>>> qes;
    std::vector<bool> vis;
    std::vector<int> ans;
    DisjointSetUnion dsu;
```

```
    void dfs(int now) {
        vis[now] = true;
        for(int nxt : tr[now]) {
            if(vis[nxt]) {
                continue;
            }
            dfs(nxt);
            dsu.merge(now, nxt);
        }
        for(auto [i, id] : qes[now]) {
            if(vis[i]) {
                ans[id] = dsu.find(i);
            }
        }
    }

   public:
    LCA(int n, int m, int s, std::vector<std::vector<int>> &&tr,
        std::vector<std::vector<std::pair<int, int>>> &&qes)
        : n(n), m(m), s(s), tr(std::move(tr)), qes(std::move(qes)), vis(n + 1), ans(m), dsu(n) {}
    std::vector<int> get_ans() {
        dfs(s);
        return ans;
    }
};
```

倍增

```
class LCA {
    int n, s, log_n;
    std::vector<std::vector<int>> tr, fa;
    std::vector<int> depth;

    void build(int now, int father, int dep) {
        depth[now] = dep;
        fa[now][0] = father;
        for (const int nxt : tr[now]) {
            if (nxt == father) {
                continue;
            }
            build(nxt, now, dep + 1);
        }
    }
public:
    LCA(int n, int s, std::vector<std::vector<int>> &&tr)
        : n(n),
          s(s),
          log_n(static_cast<int>(std::ceil(std::log2(n)))),
          tr(std::move(tr)),
          fa(n + 1, std::vector<int>(log_n + 1)),
          depth(n + 1) {
```

```
        build(s, 0, 1);

        for (int j = 1; j <= log_n; j++) {
            for (int i = 1; i <= n; i++) {
                fa[i][j] = fa[fa[i][j - 1]][j - 1];
            }
        }
    }

    int operator()(int u, int v) {
        if(depth[u] < depth[v])
            std::swap(u, v);
        for(int j = log_n; j >= 0; j--) {
            if(depth[fa[u][j]] >= depth[v])
                u = fa[u][j];
        }
        if(u == v)
            return u;
        for(int j = log_n; j >= 0; j--) {
            if(fa[u][j] != fa[v][j]) {
                u = fa[u][j];
                v = fa[v][j];
            }
        }
        return fa[u][0];
    }
};
```

Graph

Shortest Path

Floyd

```
//
// Created by 24087 on 9/19/2024.
//
#include <climits>
#include <iostream>
#include <vector>

int main() {
    int n, m, s;
    std::cin >> n >> m >> s;

    std::vector graph(n + 1, std::vector<long long>(n + 1, INT_MAX));

    for (int i = 1; i <= m; i++) {
        long long u, v, w;
        std::cin >> u >> v >> w;
        graph[u][v] = std::min(graph[u][v], w);//神金，有重边
```

```
    }
    for (int i = 1; i <= n; i++) {
        graph[i][i] = 0;
    }

    for (int k = 1; k <= n; k++) {
        for (int x = 1; x <= n; x++) {
            for (int y = 1; y <= n; y++) {
                graph[x][y] = std::min(graph[x][y], graph[x][k] + graph[k][y]);
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        std::cout << graph[s][i] << ' ';
    }
}
```

Dijkstra

```
struct edge {
    int to, val;
};
struct node {
    int idx, m_dis;
    // node(int _i, int _d) : idx(_i), m_dis(_d) {}
    friend bool operator<(const node &a, const node &b) { return a.m_dis > b.m_dis; }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m, s;
    std::cin >> n >> m >> s;

    std::vector graph(n + 1, std::vector<edge>());
    std::vector<int> dis(n + 1, INT_MAX);
    std::vector<bool> vis(n + 1);

    for (int i = 1; i <= m; i++) {
        int u, v, w;
        std::cin >> u >> v >> w;
        graph[u].push_back({v, w});
    }

    std::priority_queue<node> q;
    dis[s] = 0;

    q.push({s, 0});

    while (!q.empty()) {
```

```
        const int now = q.top().idx;
        q.pop();
        if (vis[now])
            continue;
        vis[now] = true;
        for (auto [to, val]: graph[now]) {
            if (vis[to])
                continue;
            if (dis[to] > static_cast<long long>(dis[now]) + val) {
                dis[to] = dis[now] + val;
                q.push({to, dis[to]});
                // vis[to] = true; !!!!NO!!!!!!
            }
        }
    }


    for (int i = 1; i <= n; i++) {
        std::cout << dis[i] << ' ';
    }
}
```

SPFA

```
struct edge {
    int to, val;
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m, s;
    std::cin >> n >> m >> s;

    std::vector graph(n + 1, std::vector<edge>());
    std::vector<int> dis(n + 1, INT_MAX);
    std::vector<bool> vis(n + 1);

    for (int i = 1; i <= m; i++) {
        int u, v, w;
        std::cin >> u >> v >> w;
        graph[u].push_back({v, w});
    }

    std::deque<int> q;
    dis[s] = 0;
    vis[s] = true;
    q.push_back(s);

    while (!q.empty()) {
```

```
        const int now = q.front();
        q.pop_front();
        vis[now] = false;


        for (auto [to, val]: graph[now]) {
            if (dis[to] > static_cast<long long>(dis[now]) + val) {
                dis[to] = dis[now] + val;
                if(!vis[to]) {
                    q.push_back(to);
                    vis[to] = true;
                }
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        std::cout << dis[i] << ' ';
    }
}
```

SPFA(2)

```
struct edge {
    int to, val;
};
struct node {
    int idx, m_dis;
    // node(int _i, int _d) : idx(_i), m_dis(_d) {}
    friend bool operator<(const node &a, const node &b) { return a.m_dis > b.m_dis; }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m, s;
    std::cin >> n >> m >> s;

    std::vector graph(n + 1, std::vector<edge>());
    std::vector<int> dis(n + 1, INT_MAX);

    for (int i = 1; i <= m; i++) {
        int u, v, w;
        std::cin >> u >> v >> w;
        graph[u].push_back({v, w});
    }

    std::priority_queue<node> q;
    dis[s] = 0;
```

```
    q.push({s, 0});

    while (!q.empty()) {
        const int now = q.top().idx;
        q.pop();
        for (auto [to, val]: graph[now]) {
            if (dis[to] > static_cast<long long>(dis[now]) + val) {
                dis[to] = dis[now] + val;
                q.push({to, dis[to]});
                //vis[to] = true; !!!!NO!!!!!!
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        std::cout << dis[i] << ' ';
    }
}
```

SCC

Kosaraju

```
class Kosaraju {
    int n, cnt = 0;
    std::vector<std::vector<int>> g, scc, rev, reduced;
    std::vector<int> topo, color;
    std::vector<bool> vis;

    void topo_sort(int now) {
        vis[now] = true;
        for (auto nxt : g[now]) {
            if (!vis[nxt]) {
                topo_sort(nxt);
            }
        }
        topo.push_back(now);
    }
    void dfs(int now) {
        color[now] = cnt;
        scc.back().push_back(now);
        for (auto nxt : rev[now]) {
            if (!color[nxt]) {
                dfs(nxt);
            }
        }
    }
    void calculate() {
        for (int i = 1; i <= n; i++) {
            if (!vis[i]) {
                topo_sort(i);
            }
```

```
        }
        for (auto it = topo.rbegin(); it != topo.rend(); it++) {
            if (!color[*it]) {
                scc.emplace_back();
                cnt++;
                dfs(*it);
            }
        }
    }

    void reduce() {
        reduced.resize(cnt + 1);
        for (int i = 1; i <= n; i++) {
            for (auto j : g[i]) {
                if (color[i] != color[j]) {
                    reduced[color[i]].push_back(color[j]);
                }
            }
        }
        for (auto &vec : reduced) {
            std::sort(vec.begin(), vec.end());
            vec.erase(std::unique(vec.begin(), vec.end()), vec.end());
        }
    }

public:
    Kosaraju(const int _n, std::vector<std::vector<int>> _vec)
        : n(_n),
          g(std::move(_vec)),
          scc(1),
          rev(_n + 1),
          color(_n + 1),
          vis(_n + 1) {
        for (auto &vec : g) {
            std::sort(vec.begin(), vec.end());
            vec.erase(std::unique(vec.begin(), vec.end()), vec.end());
        }
        for (auto i = 1; i <= _n; i++) {
            for (auto j : g[i]) {
                rev[j].push_back(i);
            }
        }
        calculate();
    }
    auto &get_color() { return color; }
    auto &get_topo() { return topo; }
    auto get_cnt() const { return cnt; }
    auto &get_scc() {
        for (auto &vec : scc) {
            std::sort(vec.begin(), vec.end());
        }
        return scc;
    }
```

```
    auto &get_reduced() {
        reduce();
        return reduced;
    }
};
```

Topo Sort

Kahn

```
class TopoSort {
    int n;
    std::vector<std::vector<int>> g;
    std::vector<int> in_degree, ans;
    void bfs() {
        std::queue<int> q;
        for(int i = 1; i <= n; i++) {
            if(in_degree[i] == 0) {
                q.push(i);
            }
        }
        while(!q.empty()) {
            int now = q.front();
            q.pop();
            ans.push_back(now);
            for(auto nxt : g[now]) {
                if(--in_degree[nxt] == 0) {
                    q.push(nxt);
                }
            }
        }
    }
public:
    TopoSort(int _n, std::vector<std::vector<int>> _g) : n(_n), g(std::move(_g)), in_degree(_n + 1)
{
        for(int i = 1; i <= _n; i++) {
            for(int j : g[i]) {
                in_degree[j]++;
            }
        }
    }
    auto sort() {
        bfs();
        return ans;
    }

};
```

DFS

```cpp
class TopoSort {
    int n, tot = 0;
    std::vector<std::vector<int>> g;
    std::vector<int> ans, topo_order;

    void dfs(int now) {
        for(auto i : g[now]) {
            if(topo_order[i]) {
                continue;
            }
            dfs(i);
        }
        topo_order[now] = ++tot;
        ans.push_back(now);
    }

   public:
    TopoSort(int _n, std::vector<std::vector<int>> _g)
        : n(_n), g(std::move(_g)), topo_order(_n + 1) {}

    auto sort() {
        for(int i = 1; i <= n; i++) {
            if(!topo_order[i]) {
                dfs(i);
            }
        }
        std::reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

calculate_all_topo_order

```cpp
//
// Created by 24087 on 24-11-6.
//
#include <algorithm>
#include <functional>
#include <iostream>
#include <set>
#include <vector>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n;
    std::cin >> n;
    std::vector<std::vector<int>> g(n + 1);
```

```
    std::vector<int> in_degree(n + 1), ans;

    for (int i = 1, x; i <= n; i++) {
        while (true) {
            std::cin >> x;
            if (x == 0) break;
            g[i].push_back(x);
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j : g[i]) {
            in_degree[j]++;
        }
    }

    std::function<void(int)> dfs = [&](int dep) {
        if(dep == n) {
            for(int & an : ans) {
                std::cout << an << ' ';
            }
            std::cout << '\n';
        }
        for(int i = 1; i <= n; i++) {
            if(in_degree[i] == 0) {
                in_degree[i] = -1;
                for(int j : g[i]) {
                    in_degree[j]--;
                }
                ans.push_back(i);
                dfs(dep + 1);
                ans.pop_back();
                for(int j : g[i]) {
                    in_degree[j]++;
                }
                in_degree[i] = 0;
            }
        }
    };

    dfs(0);

}
```

2025

线段树优化建图

```
#include <iostream>
#include <queue>
#include <vector>
#include <climits>
```

```cpp
#define int long long

// 线段树优化建图
class SegmentTreeGraph {
    struct Edge {
        int to, val;
    };
    int n;
    std::vector<std::vector<Edge>> tr;
    std::vector<int> idx;
    void build(const int now, const int l, const int r) {
        if (l == r) {
            idx[l] = now;
            return;
        }
        tr[now].push_back({now << 1, 0});
        tr[now].push_back({now << 1 | 1, 0});
        tr[(now << 1) + 4 * n].push_back({now + 4 * n, 0});
        tr[(now << 1 | 1) + 4 * n].push_back({now + 4 * n, 0});
        const int mid = (l + r) >> 1;
        build(now << 1, l, mid);
        build(now << 1 | 1, mid + 1, r);
    }
    void connect_point_to_range(int now, int l, int r, int L, int R, int start,
                                int val) {
        if (r < L || l > R)
            return;
        if (L <= l && r <= R) {
            //std::cerr << "l=" << l << " r=" << r << " start=" << start << " val=" << val <<
std::endl;
            tr[start + 4 * n].push_back({now, val});
            return;
        }
        int mid = (l + r) >> 1;
        connect_point_to_range(now << 1, l, mid, L, R, start, val);
        connect_point_to_range(now << 1 | 1, mid + 1, r, L, R, start, val);
    }
    void connect_range_to_point(int now, int l, int r, int L, int R, int dest,
                                int val) {
        if (r < L || l > R)
            return;
        if (L <= l && r <= R) {
            tr[now + 4 * n].push_back({dest, val});
            return;
        }
        int mid = (l + r) >> 1;
        connect_range_to_point(now << 1, l, mid, L, R, dest, val);
        connect_range_to_point(now << 1 | 1, mid + 1, r, L, R, dest, val);
    }

   public:
    explicit SegmentTreeGraph(const int _n)
```

```
        : n(_n), tr(_n << 3), idx(_n + 1) {
        build(1, 1, _n);
        for (int i = 1; i <= _n; ++i) {
            tr[idx[i]].push_back({idx[i] + 4 * _n, 0});
            tr[idx[i] + 4 * _n].push_back({idx[i], 0});
        }
    }
    void connect_point_to_range(int l, int r, int start, int val) {
        connect_point_to_range(1, 1, n, l, r, idx[start], val);
    }
    void connect_range_to_point(int l, int r, int dest, int val) {
        connect_range_to_point(1, 1, n, l, r, idx[dest], val);
    }
    void connect_point_to_point(int start, int dest, int val) {
        tr[idx[start] + 4 * n].push_back({idx[dest], val});
    }

    auto dijkstra(int s) {
        struct Dij_Node {
            int u, d;
            bool operator<(const Dij_Node &rhs) const { return d > rhs.d; }
        };
        std::vector<int> dis(8 * n, LLONG_MAX);
        std::vector<bool> vis(8 * n);
        std::priority_queue<Dij_Node> q;
        q.push({idx[s] + 4 * n, 0});
        dis[idx[s] + 4 * n] = 0;

        while (!q.empty()) {
            auto [u, d] = q.top();
            q.pop();
            if (vis[u]) continue;
            vis[u] = true;
            for (const auto &[to, val] : tr[u]) {
                if (vis[to])
                    continue;
                if (dis[to] - val> d) {
                    dis[to] = d + val;
                    q.push({to, dis[to]});
                }
            }
        }
        std::vector<int> ans(n + 1);
        for (int i = 1; i <= n; i++) {
            ans[i] = dis[idx[i] + 4 * n];
        }
        return ans;
    }
};

signed main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
```

```
    std::cout.tie(nullptr);

    int n, q, s;
    std::cin >> n >> q >> s;
    SegmentTreeGraph graph(n);
    for (int i = 1, u, v, w, l, r, opt; i <= q; i++) {
        std::cin >> opt;
        switch (opt) {
            case 1:
                std::cin >> u >> v >> w;
                graph.connect_point_to_point(u, v, w);
                break;
            case 2:
                std::cin >> u >> l >> r >> w;
                graph.connect_point_to_range(l, r, u, w);
                //graph.connect_range_to_point(l, r, u, w);
                break;
            case 3:
                std::cin >> v >> l >> r >> w;
                graph.connect_range_to_point(l, r, v, w);
                //graph.connect_point_to_range(l, r, v, w);
                break;
            default:
                std::cerr << "Man! What can I say?" << std::endl;
                break;
        }
    }
    //graph.debug_print_tr();
    auto dist = graph.dijkstra(s);
    for (int i = 1; i <= n; i++) {
        std::cout << (dist[i] == LLONG_MAX ? -1 : dist[i] ) << ' ';
    }
}
```

哈希化

```
//
// Created by guanghere on 25-8-3.
//
#include <bits/stdc++.h>

using ll = long long;
using ull = unsigned long long;

class Hash {
    inline static ull mod = 1e9 + 9;
    inline static ull base1 = 998244353;
    inline static ull base2 = 1e9 + 7;

    [[nodiscard]] static ull mul(ull a, ull b) {
        // return a % mod * (b % mod) % mod;
        return a * b % mod;
```

```
    }
    [[nodiscard]] static ull add(ull a, ull b) {
        // return (a % mod + b % mod) % mod;
        return (a + b) % mod;
    }

    [[nodiscard]] static ull power(ull base, ull exp) {
        ull res = 1;
        while (exp > 0) {
            if (exp & 1) {
                res = mul(res, base);
            }
            base = mul(base, base);
            exp >>= 1;
        }
        return res;
    }
    [[nodiscard]] static ull overflow(ull base, ull exp) {
        ull res = 1;
        while (exp > 0) {
            if (exp & 1) {
                res = res * base;
            }
            base = base * base;
            exp >>= 1;
        }
        return res;
    }

    ull h1 = 0, h2 = 0;
    Hash(const ull h1, const ull h2) : h1(h1), h2(h2) {}

public:
    Hash() = default;
    explicit Hash(const ull val)
        : h1(power(base1, val)), h2(overflow(base2, val)) {}

    Hash operator+(const Hash &other) const {
        return {add(h1, other.h1), h2 + other.h2};
    }
    Hash operator*(const Hash &other) const {
        return {mul(h1, other.h1), h2 * other.h2};
    }
    void operator+=(const Hash &other) {
        h1 = add(h1, other.h1);
        h2 += other.h2;
    }
    void operator*=(const Hash &other) {
        h1 = mul(h1, other.h1);
        h2 *= other.h2;
    }
    bool operator==(const Hash &other) const {
        return h1 == other.h1 && h2 == other.h2;
```

```
    }
};

class Segment_tree {
#define out_of_range() (r < tar_l || l > tar_r)
#define in_range() (tar_l <= l && r <= tar_r)
    struct node {
        Hash odd, even;
        Hash add_lzy = Hash(0);  // !!
    };

    int n, mod;
    std::vector<node> data;

    void pull_up(unsigned idx) {
        data[idx].odd = data[idx << 1].odd + data[idx << 1 | 1].odd;
        data[idx].even = data[idx << 1].even + data[idx << 1 | 1].even;
    }

    void make_add(unsigned idx, unsigned l, unsigned r, const Hash val) {
        // data[idx].odd +=  val * Hash(r - l + 1);
        // data[idx].even += val * Hash((r - l + 1));
        // data[idx].add_lzy += val;
        // 并非
        data[idx].odd *= val;
        data[idx].even *= val;
        data[idx].add_lzy *= val;
    }

    void push_down(unsigned idx, unsigned l, unsigned r) {
        if (data[idx].add_lzy == Hash(0)) {
            return;
        }

        auto mid = l + (r - l) / 2;
        make_add(idx << 1, l, mid, data[idx].add_lzy);
        make_add(idx << 1 | 1, mid + 1, r, data[idx].add_lzy);

        data[idx].add_lzy = Hash(0);
    }

    std::pair<Hash, Hash> ask(const unsigned idx, const unsigned l,
                              const unsigned r, const unsigned tar_l,
                              const unsigned tar_r) {
        if (out_of_range()) {
            // return {Hash(0), Hash(0)};
            return {Hash(), Hash()};  // !!
        }
        if (in_range()) {
            return {data[idx].odd, data[idx].even};
        }
        push_down(idx, l, r);
        auto mid = l + (r - l) / 2;
```

```
        auto left = ask(idx << 1, l, mid, tar_l, tar_r);
        auto right = ask(idx << 1 | 1, mid + 1, r, tar_l, tar_r);
        return {left.first + right.first, left.second + right.second};
    }

    void add(const unsigned idx, const unsigned l, const unsigned r,
             const unsigned tar_l, const unsigned tar_r, const Hash val) {
        if (out_of_range()) {
            return;
        }
        if (in_range()) {
            make_add(idx, l, r, val);
            return;
        }
        auto mid = l + (r - l) / 2;
        push_down(idx, l, r);
        add(idx << 1, l, mid, tar_l, tar_r, val);
        add(idx << 1 | 1, mid + 1, r, tar_l, tar_r, val);
        pull_up(idx);
    }

    void build_helper(const unsigned idx, const unsigned l, const unsigned r,
                      const std::vector<Hash> &nums) {
        if (l == r) {
            // data[idx].sum = nums[l - 1] % mod;
            if (l % 2 == 1) {
                data[idx].odd = nums[l - 1];
            } else {
                data[idx].even = nums[l - 1];
            }
            return;
        }
        const auto mid = l + (r - l) / 2;
        build_helper(idx << 1, l, mid, nums);
        build_helper(idx << 1 | 1, mid + 1, r, nums);
        pull_up(idx);
    };

public:
    explicit Segment_tree(const std::vector<Hash> &nums,
                          const int &p = 571373) noexcept
        : n(nums.size()), mod(p), data(nums.size() * 4 + 5) {
        build_helper(1, 1, n, nums);
    }

    std::pair<Hash, Hash> ask(const unsigned l, const unsigned r) {
        return ask(1, 1, n, l, r);
    }

    void add(const unsigned l, const unsigned r, Hash val) {
        add(1, 1, n, l, r, val);
    }
```

```
#undef out_of_range
#undef in_range
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q;
    std::cin >> n >> q;
    std::vector<Hash> nums(n);
    for (int i = 0, x; i < n; ++i) {
        std::cin >> x;
        nums[i] = Hash(x);
    }
    Segment_tree seg_tree(nums);
    while (q--) {
        int op, l, r;
        std::cin >> op >> l >> r;
        if (op == 0) {
            int val;
            std::cin >> val;
            seg_tree.add(l, r, Hash(val));
        } else {
            auto res = seg_tree.ask(l, r);
            std::cout << (res.first == res.second ? "Yes" : "No") << "\n";
        }
    }
}
```

NTT

```
//
// Created by guanghere on 25-8-7.
//
#include <algorithm>
#include <cmath>
#include <iostream>
#include <map>
#include <string>
#include <vector>

using ll = long long;

namespace ntt {
constexpr int mod = 998244353;
constexpr int G = 3;

std::map<int, std::vector<ll>> w_cache;
std::map<int, std::vector<int>> rev_cache;

ll power(ll base, ll exp) {
    ll res = 1;
```

```
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

ll modInverse(ll n) { return power(n, mod - 2); }

void transform(std::vector<ll>& a, bool invert) {
    int n = a.size();

    if (rev_cache.find(n) == rev_cache.end()) {
        int log_n = 0;
        while ((1 << log_n) < n) log_n++;
        std::vector<int> rev(n);
        for (int i = 0; i < n; i++) {
            rev[i] = 0;
            for (int j = 0; j < log_n; j++) {
                if ((i >> j) & 1) rev[i] |= 1 << (log_n - 1 - j);
            }
        }
        rev_cache[n] = std::move(rev);

        std::vector<ll> w(n);
        ll wn = power(G, (mod - 1) / n);
        w[0] = 1;
        for (int i = 1; i < n; i++) w[i] = (w[i - 1] * wn) % mod;
        w_cache[n] = std::move(w);
    }

    const auto& rev = rev_cache.at(n);
    for (int i = 0; i < n; i++) {
        if (i < rev[i]) std::swap(a[i], a[rev[i]]);
    }

    const auto& w_table = w_cache.at(n);
    for (int len = 2; len <= n; len <<= 1) {
        int step = n / len;
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j];
                ll v = (a[i + j + len / 2] * w_table[j * step]) % mod;
                a[i + j] = (u + v) % mod;
                a[i + j + len / 2] = (u - v + mod) % mod;
            }
        }
    }

    if (invert) {
        std::reverse(a.begin() + 1, a.end());
```

```
        ll n_inv = modInverse(n);
        for (ll& x : a) {
            x = (x * n_inv) % mod;
        }
    }
}
}

void solve() {
    std::string sa, sb;
    std::cin >> sa >> sb;

    if (sa == "0" || sb == "0") {
        std::cout << "0\n";
        return;
    }

    int n = sa.length();
    int m = sb.length();

    std::vector<ll> a(n), b(m);
    for (int i = 0; i < n; ++i)
        a[i] = sa[n - 1 - i] - '0';
    for (int i = 0; i < m; ++i)
        b[i] = sb[m - 1 - i] - '0';

    int len = 1;
    while (len < n + m)
        len <<= 1;

    a.resize(len, 0);
    b.resize(len, 0);

    ntt::transform(a, false);
    ntt::transform(b, false);

    std::vector<ll> c(len);
    for (int i = 0; i < len; ++i) {
        c[i] = (a[i] * b[i]) % ntt::mod;
    }

    ntt::transform(c, true);

    int res_len = n + m + 50;
    c.resize(res_len, 0);

    for (int i = 0; i < res_len - 2; ++i) {
        ll val = c[i];

        ll r = val % 2;
        if (r < 0) r += 2;

        ll q = (val - r) / 2;
```

```
        c[i] = r;
        c[i + 2] -= q;
    }

    int first_digit = res_len - 1;
    while (first_digit > 0 && c[first_digit] == 0) {
        first_digit--;
    }

    for (int i = first_digit; i >= 0; --i) {
        std::cout << c[i];
    }
    std::cout << "\n";
}

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int t;
    std::cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

三分

```
#include <iostream>
#include <vector>
#include <cmath>

class Function {
    int n;
    std::vector<double> coefficients;
public:
    Function(const int n, std::vector<double> &&vec) : n(n), coefficients(std::move(vec)) {}
    double operator()(const double x) const {
        double result = 0;
        for (int i = 0; i <= n; i++) {
            result += coefficients[i] * pow(x, i);
        }
        return result;
    }
};

double get_max_point(const Function &f, const double L, const double R) {
    constexpr double eps = 1e-7;
    double l = L, r = R;
    while (r - l > eps) {
        const double mid = (l + r) / 2;
```

```
        if (f(mid - eps) > f(mid + eps)) {
            r = mid;
        } else {
            l = mid;
        }
    }
    return l;
}

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n;
    double l, r;
    std::cin >> n >> l >> r;
    std::vector<double> coefficients(n + 1);
    for (int i = 0; i <= n; i++) {
        std::cin >> coefficients[n - i];
    }
    Function f(n, std::move(coefficients));

    std::cout << get_max_point(f, l, r) << std::endl;
}
```

tarjan

```
//
// Created by guanghere on 25-3-12.
//
#include <iostream>
#include <queue>
#include <unordered_set>
#include <vector>

class Tarjan {
    int n, timestamp = 0, cnt = 0;
    std::vector<std::vector<int>> g, sccs;
    std::vector<bool> in_stack;
    std::vector<int> low, dfn, color, stk{};
    std::vector<std::vector<int>> reduced{};

    void dfs(int now) {
        low[now] = dfn[now] = ++timestamp;
        in_stack[now] = true;
        stk.push_back(now);
        for (auto v : g[now]) {
            if (!dfn[v]) {
                dfs(v);
                low[now] = std::min(low[now], low[v]);
            } else if (in_stack[v]) {
```

```
                low[now] = std::min(low[now], dfn[v]);
            }
        }
        if (low[now] == dfn[now]) {
            int tmp;
            cnt++;
            std::vector<int> scc;
            do {
                tmp = stk.back();
                stk.pop_back();
                in_stack[tmp] = false;
                color[tmp] = cnt;
                scc.push_back(tmp);
            } while (tmp != now);
            sccs.emplace_back(std::move(scc));
        }
    }

    void reduce() {
        std::vector<std::unordered_set<int>> tmp(cnt + 1);
        for (int i = 1; i <= n; i++) {
            for (auto v : g[i]) {
                int uid = color[i];
                int vid = color[v];
                if (uid != vid) {
                    tmp[uid].insert(vid);
                }
            }
        }
        reduced.resize(cnt + 1);
        for (int i = 1; i <= cnt; i++) {
            reduced[i] = std::vector<int>(tmp[i].begin(), tmp[i].end());
        }
    }

public:
    Tarjan(int n)
        : n(n),
          g(n + 1, std::vector<int>()),
          sccs(1),
          in_stack(n + 1),
          low(n + 1),
          dfn(n + 1),
          color(n + 1) {}
    void add_edge(int u, int v) { g[u].push_back(v); }
    void run() {
        for (int i = 1; i <= n; i++) {
            if (!dfn[i]) {
                dfs(i);
            }
        }
        reduce();
    }
```

```
    int get_cnt() { return cnt; }
    const auto& get_sccs() { return sccs; }
    const auto& get_color() { return color; }
    const auto& get_reduced() { return reduced; }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int n, m;
    std::cin >> n >> m;
    Tarjan tarjan(n);

    std::vector<int> val(n + 1);
    for (int i = 1; i <= n; i++) {
        std::cin >> val[i];
    }

    for (int i = 0, u, v; i < m; i++) {
        std::cin >> u >> v;
        tarjan.add_edge(u, v);
    }
    tarjan.run();

    const int cnt = tarjan.get_cnt();
    const auto &sccs = tarjan.get_sccs();
    const auto &color = tarjan.get_color();
    const auto g = tarjan.get_reduced();
    //std::cerr << cnt << std::endl;

    //topo
    std::vector<int> in_degree(cnt + 1), dp(cnt + 1), sum(cnt + 1);
    std::queue<int> q;
    for (int i = 1; i <= cnt; i++) {
        for (auto v : g[i]) {
            in_degree[v]++;
        }
    }
    for (int i = 1; i <= cnt; i++) {
        for (const auto now : sccs[i]) {
            sum[i] += val[now];
        }
        if (in_degree[i] == 0) {
            q.push(i);
            dp[i] = sum[i];
        }
    }

    while (!q.empty()) {
        int now = q.front();
        q.pop();
```

```
        for (auto v : g[now]) {
            in_degree[v]--;
            dp[v] = std::max(dp[v], dp[now] + sum[v]);
            if (in_degree[v] == 0) {
                q.push(v);
            }
        }
    }
    int ans = 0;
    for (int i = 1; i <= cnt; i++) {
        ans = std::max(ans, dp[i]);
    }
    std::cout << ans << '\n';
}
```

Treap

```
//
// Created by guanghere on 25-7-16.
//
#include <bits/stdc++.h>

class Treap {
    struct Node {
        Node *ch[2];
        int val, cnt, siz;
        unsigned rank;
        explicit Node(const int v)
            : ch{nullptr, nullptr}, val(v), cnt(1), siz(1), rank(gen()) {
        }
    };
    enum RotType { L = 1, R = 0 };
    Node *root = nullptr;
    inline static std::random_device rd;
    inline static std::mt19937 gen = std::mt19937(rd());

    static void update(Node *const now) {
        if (!now) return;
        now->siz = now->cnt + (now->ch[0] ? now->ch[0]->siz : 0) +
                   (now->ch[1] ? now->ch[1]->siz : 0);
    }

    static void rotate(Node *&now, const RotType type) {
        Node *tmp = now->ch[type];
        now->ch[type] = tmp->ch[1 ^ type];
        tmp->ch[1 ^ type] = now;
        update(now);
        update(tmp);
        now = tmp;
    }

    static void insert(Node *&now, const int x) {
```

```
        if (now == nullptr) {
            now = new Node(x);
            return;
        }
        if (x == now->val) {
            now->cnt++;
        } else if (x < now->val) {
            insert(now->ch[0], x);
            if (now->ch[0]->rank > now->rank) {
                rotate(now, R);
            }
        } else {
            insert(now->ch[1], x);
            if (now->ch[1]->rank > now->rank) {
                rotate(now, L);
            }
        }
        update(now);
    }
    static void erase(Node *&now, const int x) {
        if (now == nullptr) return;
        if (x < now->val) {
            erase(now->ch[0], x);
        } else if (x > now->val) {
            erase(now->ch[1], x);
        } else {
            if (now->cnt > 1) {
                now->cnt--;
            } else {
                if (now->ch[0] == nullptr || now->ch[1] == nullptr) {
                    Node *tmp = now->ch[0] ? now->ch[0] : now->ch[1];
                    delete now;
                    now = tmp;
                } else {
                    if (now->ch[0]->rank > now->ch[1]->rank) {
                        rotate(now, R);
                        erase(now->ch[1], x);
                    } else {
                        rotate(now, L);
                        erase(now->ch[0], x);
                    }
                }
            }
        }
        update(now);
    }

    static int count_less(const Node *now, const int x) {
        if (now == nullptr) return 0;
        if (x > now->val) {
            return (now->ch[0] ? now->ch[0]->siz : 0) + now->cnt +
                   count_less(now->ch[1], x);
        }
```

```
        if (x == now->val) {
            return now->ch[0] ? now->ch[0]->siz : 0;
        }
        // x < now->val
        return count_less(now->ch[0], x);
    }

    static int find_by_rank(const Node *now, const int x) {
        int less_siz = now->ch[0] ? now->ch[0]->siz : 0;
        if (x <= less_siz) {
            return find_by_rank(now->ch[0], x);
        }
        if (x > less_siz + now->cnt) {
            return find_by_rank(now->ch[1], x - less_siz - now->cnt);
        }
        return now->val;
    }

    static int pre(const Node *now, const int x) {
        if (now == nullptr) return None;
        if (x < now->val) {
            return pre(now->ch[0], x);
        }
        if (x == now->val) {
            if (now->ch[0] == nullptr) {
                return None;  // 没有前驱
            }
            const Node *tmp = now->ch[0];
            while (tmp->ch[1]) {
                tmp = tmp->ch[1];
            }
            return tmp->val;
        }
        // x > now->val
        if (now->ch[1] == nullptr) {
            return now->val;  // 没有后继，返回当前值
        }
        int res = pre(now->ch[1], x);
        return (res == None) ? now->val : res;  // 返回前驱或当前值
    }
    static int suf(const Node *now, const int x) {
        if (now == nullptr) return None;
        if (x < now->val) {
            if (now->ch[0] == nullptr) {
                return now->val;
            }
            int res = suf(now->ch[0], x);
            return (res == None) ? now->val : res;  // 返回后继或当前值
        }
        if (x == now->val) {
            if (now->ch[1] == nullptr) {
                return None;  // 没有后继
            }
```

```
            const Node *tmp = now->ch[1];
            while (tmp->ch[0]) {
                tmp = tmp->ch[0];
            }
            return tmp->val;
        }
        // x > now->val
        return suf(now->ch[1], x);
    }

   public:
    static constexpr int None = INT_MAX;
    Treap() = default;
    ~Treap() {
        std::function<void(Node *)> clear = [&](const Node *now) {
            if (now == nullptr) return;
            clear(now->ch[0]);
            clear(now->ch[1]);
            delete now;
        };
        clear(root);
        root = nullptr;
    }
    void insert(const int x) { insert(root, x); }
    void erase(const int x) { erase(root, x); }
    [[nodiscard]] int rank(const int x) const {
        return count_less(root, x) + 1;
    }
    [[nodiscard]] int find_by_rank(const int x) const {
        if (x <= 0 || x > (root ? root->siz : 0)) {
            std::cerr << "Error: find_by_rank called with invalid rank: " << x
                      << ".\n";
            return None;
        }
        return find_by_rank(root, x);
    }
    [[nodiscard]] int pre(const int x) const {
        return pre(root, x);
    }
    [[nodiscard]] int suf(const int x) const {
        return suf(root, x);
    }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int n;
    std::cin >> n;
    Treap treap;
    for (int i = 0; i < n; ++i) {
        int op, x;
```

```
        std::cin >> op >> x;
        if (op == 1) {
            treap.insert(x);
        } else if (op == 2) {
            treap.erase(x);
        } else if (op == 3) {
            std::cout << treap.rank(x) << "\n";
        } else if (op == 4) {
            std::cout << treap.find_by_rank(x) << "\n";
        } else if (op == 5) {
            std::cout << treap.pre(x) << "\n";
        } else if (op == 6) {
            std::cout << treap.suf(x) << "\n";
        }
    }
}
```